

Fiche 319 — Écrire un paquet : structure, DESCRIPTION, NAMESPACE, check

Matière	Code · Langage R
Cours source	R Core Team, <i>Writing R Extensions</i> 4.6.1 — chapitre 1 « Creating R packages » (§1.1 structure, §1.1.1 DESCRIPTION , §1.1.3 dépendances, §1.1.5 sous-répertoires, §1.3 vérifier et construire, §1.5 espaces de noms) et chapitre 2 « Writing R documentation files » (§2.1 format Rd)
Difficulté	Avancé — beaucoup de conventions, peu de concepts
Temps d'étude estimé	1 h 45
Prérequis	Fiches 309, 310, 311, 315, 317
Concepts clés	paquet source , tarball , paquet installé , paquet binaire , paquet ≠ library , installer / construire / vérifier, chargement paresseux , DESCRIPTION au format Debian Control File , les sept champs obligatoires , Depends / Imports / Suggests / Enhances , NAMESPACE scellé , export / exportPattern / import / importFrom , S3method , .onLoad / .onAttach , useDynLib , R CMD check , R CMD build , format Rd
À retenir en priorité	Un paquet n'est pas une bibliothèque · la version n'est pas un nombre décimal · les trois règles Depends / Imports · S3method dispense d'exporter · check et build tournent en --vanilla .

Vue d'ensemble

[illegible]

QUATRE OPERATIONS	install	R CMD INSTALL / install.packages()
	build	creee le tarball, nettoie, fabrique la doc des vignettes
	check	installe, teste, verifie la coherence et la portabilite
	compile	!! terme INEXACT pour un paquet -- c'est le CODE C/C++/ Fortran qui se compile, ou le BYTE-COMPILE du code R

!! UN PAQUET N'EST PAS UNE « LIBRARY »

library = 1. le REPERTOIRE ou les paquets sont installes
2. la bibliotheque partagee du systeme (DLL, .so, .dylib)

STRUCTURE MINIMALE DESCRIPTION NAMESPACE

sous-repertoires R data demo exec inst man po src tests tools
vignettes (absents possibles, VIDES interdits)

DESCRIPTION format « Debian Control File » ; noms de champs SENSIBLES A LA CASSE

OBLIGATOIRES Package Version License Description Title Author Maintainer

!! Version n'est PAS un decimal : 0.9 < 0.75 car 9 < 75

DEPENDANCES Depends attache AVANT le paquet

Imports espace de noms importe, PAS attache <- le cas normal

Suggests exemples, demos, tests, vignettes

Enhances paquets « enrichis » par le votre

« A package should be listed in ONLY ONE of these fields. »

NAMESPACE export(f, g) exportPattern("^[^.]")

import(foo) importFrom(foo, f, g)

S3method(print, foo) <- pas besoin d'exporter print.foo

useDynLib(foo)

!! « Namespaces are SEALED once they are loaded. »

DOCUMENTATION format Rd -- en-tete (OBLIGATOIRE) + corps + pied de mots-cles

Le problème posé. « *Packages provide a mechanism for loading optional code, data and documentation as needed. The R distribution itself includes about 30 packages.* » (§1)

INTUITION — CE CHAPITRE EST UN CONTRAT.

La fiche 315 décrivait les paquets **du point de vue de l'utilisateur** — « *Here, we will describe them from a user's point of view* », disait le §13. Le présent chapitre décrit **ce qu'il faut fournir** pour qu'un paquet fonctionne : une structure de répertoires, deux fichiers de métadonnées, un format de documentation. **Presque tout y est convention** ; la difficulté n'est pas conceptuelle, elle est dans le nombre de règles.

« In the following, **we assume that you know the** `library()` **command, including its** `lib.loc` **argument, and we also assume basic knowledge of the** `R CMD INSTALL` **utility.** » — les prérequis sont explicites (fiches 300 et 315).

● Concept 1 — Le vocabulaire, et le contresens à éviter

DÉFINITION (§1).

« **A package is a directory of files which extend R, a source package** (the master files of a package), or **a tarball containing the files of a source package, or an installed package**, the result of running `R CMD INSTALL` on a source package. On some platforms ... there are also **binary packages**, a zip file or tarball containing the files of an installed package **which can be unpacked rather than installing from sources.** »

Forme	Ce que c'est
paquet source	« the master files of a package »
tarball	l'archive du paquet source, prête à distribuer
paquet installé	« the result of running <code>R CMD INSTALL</code> on a source package »
paquet binaire	« can be unpacked rather than installing from sources » — macOS, Windows x86_64

Le contresens (§1). « *A package is not a library. The latter is used in two senses in R documentation* » :

1. « **A directory into which packages are installed**, e.g. `/usr/Lib/R/Library` : in that sense it is sometimes referred to as **a library directory or library tree** » ;
2. *« **That used by the operating system, as a shared, dynamic or static library or (especially on Windows) a DLL**, where the second L stands for "library". »*

(note 1 : « **although this is a persistent mis-usage. It seems to stem from S**, whose analogues of R's packages were officially known as **library sections** and later as **chapters**, but almost always referred to as **libraries**. »)

L'erreur est donc historique et tenace — d'où l'ironie que la commande de chargement s'appelle `library()` .

« macOS distinguishes between **shared objects (extension `.so`)** and **dynamic libraries (extension `.dyLib`)** », là où ailleurs « *these concepts are interchangeable* ».

1.1 Les opérations

DÉFINITION (§1).

« There are **a number of well-defined operations on source packages**. »

Opération	Ce qu'elle fait
installation	« the most common ... using <code>R CMD INSTALL</code> or <code>install.packages</code> »
build	« taking a source directory and creating a tarball ready for distribution , including cleaning it up and creating PDF/HTML documentation from any vignettes »
check	« a test installation is done and tested (including running its examples) ; also, the contents of the package are tested in various ways for consistency and portability »
compilation	« Compilation is not a correct term for a package. »

Le point sur la compilation (§1). « *Installing a source package which contains C, C++ or Fortran code will involve compiling that code. There is also the possibility of "byte" compiling the R code in a package (using the facilities of package compiler): nowadays this is enabled by default for all packages. So compiling a package may come to mean byte-compiling its R code.* »

Charger et attacher (§1). « *It used to be unambiguous to talk about loading an installed package using `library()`, but since the advent of package namespaces this has been less clear: people now often talk about loading the package's namespace and then attaching the package so it becomes visible on the search path. Function `library` performs both steps, but a package's namespace can be loaded without the package being attached (for example by calls like `splines::ns`).* »

C'est l'explication exacte du `loadedNamespaces()` de la fiche 315 : « *some packages may be loaded but not available on the search list* ».

Le chargement paresseux (§1). « *This is part of the installation, always selected for R code but optional for data. When used, the R objects of the package are created at installation time and stored in a database in the `R` directory of the installed package, being loaded into the session at first use. This makes the R session start up faster and use less (virtual) memory.* »

● Concept 2 — La structure d'un paquet

DÉFINITION (§1.1).

« *The sources of an R package consist of a subdirectory containing the files `DESCRIPTION` and `NAMESPACE`, and the subdirectories `R`, `data`, `demo`, `exec`, `inst`, `man`, `po`, `src`, `tests`, `tools` and `vignettes` (some of which can be missing, but which should not be empty).* »

« The package subdirectory may also contain files `INDEX`, `configure`, `cleanup`, `LICENSE`, `LICENCE` and `NEWS`. Other files such as `INSTALL`, `README` / `README.md`, or `ChangeLog` will be ignored by R, but may be useful to end users. »

« **Except where specifically mentioned, packages should not contain Unix-style "hidden" files/directories** (that is, those whose name starts with a dot). » — les exceptions étant `.Rbuildignore` , `.Rinstignore` et `vignettes/.install_extras` .

MÉTHODE (§1.1).

« **The R function** `package.skeleton` **can help to create the structure for a new package** : see its help page for details. »

2.1 Les contraintes sur les noms

Règle (§1.1). *« **The package subdirectory should be given the same name as the package. Because some file systems (e.g., those on Windows and by default on macOS) are not case-sensitive, to maintain portability it is strongly recommended that case distinctions not be used to distinguish different packages.** For example, if you have a package named `foo` , do not also create a package named `Foo` . »*

C'est le premier piège de portabilité de la fiche 318, appliqué aux noms de paquets.

Règle (§1.1). « *To ensure that file names are valid across file systems and supported operating systems, the ASCII control characters as well as the characters `"` , `*` , `:` , `/` , `<` , `>` , `?` , `\` , and `|` are not allowed in file names. In addition, files with names `con` , `prn` , `aux` , `cLock$` , `nul` , `com1` to `com9` , and `Lpt1` to `Lpt9` after conversion to lower case and stripping possible "extensions" are disallowed. Also, file names in the same directory must not differ only by case.* »

« In addition, the basenames of `.Rd` files may be used in URLs and so must be ASCII and not contain `%` . For maximal portability filenames should only contain ASCII characters ... we exclude space as many utilities do not accept spaces in file paths ... It would be good practice to avoid the shell metacharacters `(){}'[]$~` . »

« **Packages are normally distributed as tarballs, and these have a limit on path lengths ... for maximal portability, 100 bytes.** »

Pas de binaires (§1.1). « *A source package if possible should not contain binary executable files : they are not portable, and a security risk if they are of the appropriate architecture.* `R CMD check` will warn about them unless they are listed ... in a file `BinaryFiles` . **Note that CRAN will not accept submissions containing binary files even if they are listed.** »

● Concept 3 — Le fichier DESCRIPTION

DÉFINITION (§1.1.1).

« *The DESCRIPTION file contains basic information about the package.* » L'exemple du manuel :

```
Package: pkgname
Version: 0.5-1
Date: 2015-01-01
Title: My First Collection of Functions
Authors@R: c(person("Joe", "Developer", role = c("aut", "cre"),
                  email = "Joe.Developer@some.domain.net",
                  comment = c(ORCID = "nnnn-nnnn-nnnn-nnnn")),
            person("Pat", "Developer", role = "aut"),
            person("A.", "User", role = "ctb",
                  email = "A.User@whereever.net"))
Author: Joe Developer [aut, cre],
       Pat Developer [aut],
       A. User [ctb]
Maintainer: Joe Developer <Joe.Developer@some.domain.net>
Depends: R (>= 3.1.0), nlme
Suggests: MASS
Description: A (one paragraph) description of what
            the package does and why it may be useful.
License: GPL (>= 2)
URL: https://www.r-project.org, http://www.another.url
BugReports: https://pkgname.bugtracker.url
```

Le format (§1.1.1). « *The format is that of a version of a "Debian Control File" ... Fields start with an ASCII name immediately followed by a colon : the value starts after the colon and a space. Continuation lines (for example, for descriptions longer than one line) start with a space or tab. Field names are case-sensitive : all those used by R are capitalized.* »

« *For maximal portability, the `DESCRIPTION` file should be written entirely in ASCII — if this is not possible it must contain an `Encoding` field.* »

3.1 Les sept champs obligatoires

Règle (§1.1.1). « *The `Package` , `Version` , `License` , `Description` , `Title` , `Author` , and `Maintainer` fields are mandatory, all other fields are optional. Fields `Author` and `Maintainer` can be auto-generated from `Authors@R` , and may be omitted if the latter is provided.* »

Champ	Contrainte
<code>Package</code>	« <i>only (ASCII) letters, numbers and dot, have at least two characters and start with a letter and not end in a dot</i> »
<code>Version</code>	« <i>a sequence of at least two (and usually three) non-negative integers separated by single <code>.</code> or <code>-</code></i> »
<code>License</code>	traité au §1.1.2
<code>Title</code>	« <i>a short description ... may truncate the title to 65 characters ... use title case ... not use any markup, not have any continuation lines, and not end in a period ... Do not repeat the package name</i> »
<code>Description</code>	« <i>a comprehensive description ... several (complete) sentences, but only one paragraph ... It is good practice not to start with the package name, "This package" or similar ... URLs should be enclosed in angle brackets</i> »
<code>Author</code>	« <i>a plain text field intended for human readers, but not for automatic processing ... all significant contributors must be included</i> »
<code>Maintainer</code>	« <i>a single name followed by a valid (RFC 2822) email address in angle brackets ... For a CRAN package it should be a person, not a mailing list and not a corporate entity</i> »

Le piège de la version (§1.1.1). *« It is not a decimal number, so for example `0.9 < 0.75` since `9 < 75` . »* Et : « *a version such as `0.01` or `0.01.0` will be handled as if it were `0.1-0` ».*

Chaque composante est un entier comparé séparément. La version 0.9 vient **avant** la 0.75 — l'inverse de l'intuition arithmétique. C'est la même convention que pour les numéros de version de logiciels en général, et elle surprend à chaque fois.

Deux conventions typographiques du manuel : « *Refer to other packages and external software in single quotes, and to book titles (and similar) in double quotes.* » — pour `Title` comme pour `Description` .

● Concept 4 — Les quatre champs de dépendance

DÉFINITION (§1.1.3).

« *The `Depends` field gives a comma-separated list of package names which this package depends on. Those packages will be attached before the current package when `Library` or `require` is called. Each package name may be optionally followed by a comment in parentheses specifying a version requirement* — e.g. `MASS (>= 3.1-20)` . »

Champ	Rôle
<code>Depends</code>	« <i>attached before the current package</i> »
<code>Imports</code>	« <i>packages whose namespaces are imported from ... but which do not need to be attached</i> »
<code>Suggests</code>	« <i>packages that are not necessarily needed ... used only in examples, demos, tests or vignettes, and packages loaded in the body of functions</i> »
<code>Enhances</code>	« <i>packages "enhanced" by the package at hand, e.g. by providing methods for classes from these packages</i> »

Les précisions sur `Imports` (§1.1.3). « *Namespaces accessed by the `::` and `:::` operators must be listed here, or in `Suggests` or `Enhances`. Ideally this field will include all the standard packages that are used, and it is important to include S4-using packages (as their class definitions can change ...). Packages declared in the `Depends` field should not also be in the `Imports` field.* »

Les trois règles générales (§1.1.3).

1. « **A package should be listed in only one of these fields.** »
2. « **Packages whose namespace only is needed to load the package using `Library(pkgname)` should be listed in the `Imports` field and not in the `Depends` field. Packages listed in `import` or `importFrom` directives in the `NAMESPACE` file should almost always be in `Imports` and not `Depends`.** »
3. « **Packages that need to be attached to successfully load the package using `Library(pkgname)` must be listed in the `Depends` field.** »

EN CLAIR — LA RÈGLE DE DÉCISION.

La question à se poser est : « *l'utilisateur de mon paquet doit-il **voir** les fonctions de cet autre paquet ?* » Si **non** — le cas de loin le plus fréquent —, c'est `Imports`. Si **oui**, c'est `Depends`. Les règles 2 et 3 disent exactement cela, et la règle 1 interdit de couvrir ses arrières en le mettant partout.

La dépendance à R lui-même (§1.1.3) : *« if the package works only with R version 4.0.0 or later, include `R (>= 4.0)` in the `Depends` field. (As here, trailing zeroes can be dropped and it is recommended that they are.) »*

« *It makes no sense to declare a dependence on R without a version specification, nor on the package `base` : this is an R package and package `base` is always available.* » Et : « *It is inadvisable to use a dependence on R with patch level (the third digit) other than zero.* »

« **Both `Library` and the R package checking facilities use this field : hence it is an error to use improper syntax or misuse the `Depends` field for comments on other software that might be needed.** »

● Concept 5 — Les sous-répertoires

Règle — le répertoire `R` (§1.1.5). « **The `R` subdirectory contains R code files, only. The code files ... must start with an ASCII letter or digit and have one of the extensions `.R`, `.S`, `.q`, `.r`, or `.s`. We recommend using `.R`. It should be possible to read in the files using `source()`, so R objects must be created by assignments. Note that there need be no connection between the name of the file and the R objects created by it.** »

« **Ideally, the R code files should only directly assign R objects and definitely should not call functions with side effects such as `require` and `options`.** »

« **Extreme care is needed if top-level computations are made to depend on availability or not of other packages ... Nor should they depend on the availability of external resources such as downloads.** »

Deux exceptions : `R/sysdata.rda` — « **lazy-loaded into the namespace environment** — this is intended for **system datasets that are not intended to be user-accessible via `data`** » — et les fichiers en `.in`, pour un script `configure`.

Règle — l'encodage (§1.1.5). « **Only ASCII characters (and the control characters `tab`, `form feed`, `LF` and `CR`) should be used in code files. Other characters are accepted in comments, but then the comments may not be readable in e.g. a UTF-8 locale. Non-ASCII characters in object names will normally fail when the package is installed. Any byte will be allowed in a quoted character string but `\uxxxx` escapes should be used for non-ASCII characters.** »

Règle — le répertoire `man` (§1.1.5). *« The `man` subdirectory should contain (only) documentation files for the objects in the package in R documentation (Rd) format. ... the names must be valid in `file://` URLs, which means they must be entirely ASCII and not contain `% .` »*

« Note that all user-level objects in a package should be documented ; if a package `pkg` contains user-level objects which are for "internal" use only, it should provide a file `pkg-internal.Rd` which documents all such objects, and clearly states that these are not meant to be called by the user. Note that packages which use internal objects extensively should not export those objects from their namespace, when they do not need to be documented. »

« Having a `man` directory containing no documentation files may give an installation error. »

Règle — le répertoire `src` (§1.1.5). *« The sources and headers for the compiled code are in `src` , plus optionally a file `Makevars` or `Makefile` . When a package is installed using `R CMD INSTALL` , `make` is used to control compilation and linking into a shared object ... providing support for C, C++, fixed- or free-form Fortran, Objective C and Objective C++ with extensions `.c` , `.cc` or `.cpp` , `.f` , `.f90` or `.f95` , `.m` , and `.mm` . We recommend using `.h` for headers. »* — voir fiche 320.

*« The `R` and `man` subdirectories may contain OS-specific subdirectories named `unix` or `windows` .
»*

● Concept 6 — Le fichier `NAMESPACE`

DÉFINITION (§1.5).

« *R has a namespace management system for code in packages. This system allows the package writer to specify which variables in the package should be exported to make them available to package users, and which variables should be imported from other packages.* »

« *The namespace for a package is specified by the `NAMESPACE` file in the top level package directory. Note that although the file looks like R code (and often has R-style comments) it is not processed as R code. Only very simple conditional processing of `if` statements is implemented.* »

Le scellement (§1.5). « *Namespaces are sealed once they are loaded. Sealing means that imports and exports cannot be changed and that internal variable bindings cannot be changed. Sealing allows a simpler implementation strategy ... and allows code analysis and compilation tools to accurately identify the definition corresponding to a global variable reference in a function body.* »

C'est la contrepartie de la garantie de la fiche 315 : si un paquet ne peut pas être perturbé de l'extérieur, c'est parce que son espace de noms est **verrouillé**.

La stratégie de recherche (§1.5). « *If not found locally, R searches the package namespace first, then the imports, then the base namespace and then the normal search path (so the base namespace precedes the normal search rather than being at the end of it).* »

C'est la chaîne de la fiche 310 (*R Language Definition* §3.5.4), **redonnée du côté de l'auteur**.

6.1 Les directives

Directive (§1.5.1)	Effet
<code>export(f, g)</code>	« specifies that the variables <code>f</code> and <code>g</code> are to be exported » — « variable names may be quoted, and reserved words and non-standard names such as <code>[<-.fractions</code> must be »
<code>exportPattern("^[^.]")</code>	« exports all variables that do not start with a period » — une expression régulière POSIX étendue
<code>import(foo, bar)</code>	« all exported variables in the packages <code>foo</code> and <code>bar</code> are to be imported »
<code>importFrom(foo, f, g)</code>	« the exported variables <code>f</code> and <code>g</code> of the package <code>foo</code> »

La recommandation (§1.5.1). « Using `importFrom` selectively rather than `import` is good practice and recommended notably when importing from packages with more than a dozen exports and especially from those written by others (so what they export can change in future). »

Et sur `exportPattern` : « **such broad patterns are not recommended for production code : it is better to list all exports or use narrowly-defined groups ... Beware of patterns which include names starting with a period : some of these are internal-only variables and should never be exported.** »

« **Packages implicitly import the base namespace.** »

6.2 Enregistrer les méthodes S3

PROBLÈME (§1.5.2).

« *The standard method for S3-style `UseMethod` dispatching might fail to locate methods defined in a package that is imported but not attached to the search path. To ensure that these methods are available the packages defining the methods should ensure that the generics are imported and register the methods using `S3method` directives.* »

```
S3method(print, foo)
```

« *ensures that the method is registered and available for `UseMethod` dispatch, and the function `print.foo` does not need to be exported. Since the generic `print` is defined in `base` it does not need to be imported explicitly.* »

C'est le complément indispensable de la fiche 311. Le dispatch cherche `generic.class` par le nom ; si le paquet n'est pas attaché, le nom n'est visible nulle part. `S3method` inscrit la méthode dans un registre — et permet de la garder **non exportée**, ce qui est exactement ce que recommandait le §1.1.5 pour les objets internes.

« *It is possible to specify a third argument to `S3method`, the function to be used as the method* », par exemple `S3method(print, check_so_symbols, .print.via.format)`.

« *As from R 3.6.0 one can also use `S3method()` directives to perform **delayed registration*** » — `S3method(pkg::gen, cls)` dans un `if`, pour n'enregistrer que **quand l'espace de noms de `pkg` est chargé**.

6.3 Les points d'entrée

DÉFINITION (§1.5.3).

« *There are a number of hooks called as packages are loaded, attached, detached, and unloaded. Since loading and attaching are distinct operations, separate hooks are provided for each. These hook functions are called `.onLoad` and `.onAttach`. They both take arguments `Libname` and `pkgname` ; they should be defined in the namespace but not exported.* »

Moment	Fonction
chargement de l'espace de noms	<code>.onLoad</code>
attachement au chemin de recherche	<code>.onAttach</code>
détachement	<code>.onDetach</code> ou <code>.Last.lib</code> (celle-ci doit être exportée)
déchargement de l'espace de noms	<code>.onUnload</code>

Les erreurs fréquentes, énumérées par le manuel (§1.5.3). « *These hooks are often used incorrectly. People forget to export `.Last.lib`. Compiled code should be loaded in `.onLoad` (or via a `useDynLib` directive) and unloaded in `.onUnload`. Do remember that a package's namespace can be loaded without the namespace being attached (e.g. by `pkgname::fun`) and that a package can be detached and re-attached whilst its namespace remains loaded.* »

« *Packages are not likely to need `.onAttach` (except perhaps for a start-up banner) ; code to set options and load shared objects should be placed in a `.onLoad` function.* »

« *It is good practice for these functions to be quiet. Any messages should use `packageStartupMessage` so users (including check scripts) can suppress them if desired.* »

Et `useDynLib` : « A `NAMESPACE` file can contain one or more `useDynLib` directives which allows shared objects that need to be loaded. »

Le paquet `foo`, fichier `foo.R` :

```
x <- 1
f <- function(y) c(x, y)
foo <- function(x) .Call("foo", x, PACKAGE = "foo")
print.foo <- function(x, ...) cat("<a foo>\n")
```

et son `NAMESPACE` :

```
useDynLib(foo)
export(f, foo)
```



```
S3method(print, foo)
```

Le paquet `bar`, fichier `bar.R` :

```
c <- function(...) sum(...)
g <- function(y) f(c(y, 7))
h <- function(y) y + 9
```

et son `NAMESPACE` :

```
import(foo)
export(g, h)
```

Le résultat annoncé par le manuel :

```
> g(6)
[1] 1 13
```

Étape 1 — ce que fait `library(bar)`. « **Calling `library(bar)` loads `bar` and attaches its exports to the search path. Package `foo` is also loaded but not attached to the search path.** » — l' `import(foo)` **charge** `foo` sans le rendre visible (concept 1).

Étape 2 — dérouler `g(6)`. `g <- function(y) f(c(y, 7))`. On évalue d'abord `c(6, 7)`.

Étape 3 — quel `c` ? **On est dans le corps de `g`, donc dans l'espace de noms de `bar`**, où `c <- function(...) sum(...)`. La recherche commence **par le paquet lui-même** (§1.5). Donc `c(6, 7)` vaut `sum(6, 7) = 13`.

Étape 4 — appeler `f(13)`. `f` vient de `foo` : `f <- function(y) c(x, y)`.

Étape 5 — quel `c`, cette fois ? **On est maintenant dans l'espace de noms de `foo`**, qui **ne définit pas** `c`. La recherche continue : imports de `foo`, puis `base` — où `c` est la concaténation. Donc `c(x, 13)` avec `x <- 1` de `foo`, soit `c(1, 13)`.

Étape 6 — le résultat. `[1] 1 13`.

Étape 7 — ce que le manuel en conclut. « **This is consistent with the definitions of `c` in the two settings : in `bar` the function `c` is defined to be equivalent to `sum`, but in `foo` the variable `c` refers to the standard function `c` in `base`.** »

Étape 8 — la leçon. **Chaque fonction cherche ses noms depuis son propre paquet.** Redéfinir `c` dans `bar` ne perturbe **ni** `foo`, **ni** `base` — c'est exactement la garantie de la fiche 315 : « *objects cannot be masked by objects of the same name in the global environment or in other packages* ». Et cela vaut **entre paquets**, pas seulement contre l'utilisateur.

Étape 9 — remarquer aussi. `print.foo` **n'est pas exportée** : seule la directive `S3method(print, foo)` la rend accessible au dispatch. C'est le patron recommandé du §1.5.2.

● Concept 7 — Vérifier et construire

La note préliminaire (§1.3). *« `R CMD check` and `R CMD build` run R processes with `--vanilla` in which none of the user's startup files are read. If you need `R_LIBS` set (to find packages in a non-standard library) you can set it in the environment : also you can use the check and build environment files (`R_CHECK_ENVIRON` et `R_BUILD_ENVIRON` ; à défaut `~/.R/check.Renviron` et `~/.R/build.Renviron`). »*

C'est l'explication de la mise en garde de la fiche 300 : « `R CMD` does not of itself use any R startup files ». Le « **ça marche chez moi** » vient de là, et le manuel donne ici **la solution** : passer par ces fichiers d'environnement dédiés.

« You may need to set the environment variable `TMPDIR` to point to a suitable writable directory with a path not containing spaces — use forward slashes for the separators. Also, the directory needs to be on a case-honouring file system. »

DÉFINITION (§1.3.1).

« Using `R CMD check`, the R package checker, one can test whether source R packages work correctly. It can be run on one or more directories, or compressed package tar archives. **It is strongly recommended that the final checks are run on a tar archive prepared by `R CMD build`.** »

Ce que `check` vérifie — extrait de la liste du §1.3.1, dans l'ordre :

#	Vérification
1	« The package is installed. This will warn about missing cross-references and duplicate aliases in help files »
2	« The file names are checked to be valid across file systems and supported operating system platforms »
4	« The files are checked for binary executables »
5	« The <code>DESCRIPTION</code> file is checked for completeness ... all packages mentioned in <code>library</code> or <code>requires</code> or from which the <code>NAMESPACE</code> file imports or are called via <code>::</code> or <code>:::</code> are listed »
8	« The R files are checked for syntax errors. Bytes which are non-ASCII are reported as warnings, but these should be regarded as errors »
9	« it is checked that the package can be loaded ... first with the usual default packages and then only with package <code>base</code> already loaded »
10	« The R code is checked ... using <code>codetools</code> ... whether S3 methods have all the arguments of the corresponding generic, and whether the final argument of replacement functions is called <code>value</code> ... all foreign function calls ... are tested to see if they have a <code>PACKAGE</code> argument »
11	« The Rd files are checked for correct syntax and metadata, including the presence of the mandatory fields (<code>\name</code> , <code>\alias</code> , <code>\title</code> and <code>\description</code>) »
12	« A check is made for missing documentation entries, such as undocumented user-level objects »
14	« whether all function arguments given in <code>\usage</code> sections ... are documented in the corresponding <code>\arguments</code> section »
16	« C, C++ and Fortran source and header files are tested for portable (LF-only) line endings »

CONTRÔLE — DEUX POINTS DE CETTE LISTE RECOUPENT DES FICHES PRÉCÉDENTES.

Le point 10 vérifie que « **the final argument of replacement functions is called** `value` » — c'est exactement la règle du §3.4.4 (fiche 302). Et « **S3 methods have all the arguments of the corresponding generic** » — la règle de la fiche 311, ici **contrôlée automatiquement**.

DÉFINITION (§1.3.2).

« Using `R CMD build`, the R package builder, one can build R package tarballs from their sources. It is recommended that packages are built for release by the current release version of R or "r-patched", to avoid inadvertently picking up new features of a development version of R. »

« Prior to actually building the package ... it is tested whether object indices exist and can be assumed to be up-to-date, and C, C++ and Fortran source files ... are tested and converted to LF line-endings if necessary. »

« Run-time checks whether the package works correctly should be performed using `R CMD check` prior to invoking the final build procedure. »

Exclure des fichiers (§1.3.2). « To exclude files from being put into the package, one can specify a list of exclude patterns in file `.Rbuildignore` in the top-level source directory. These patterns should be Perl-like regular expressions, one per line, to be matched case-insensitively.

» Sont exclus **par défaut** : les répertoires de contrôle de version, ceux nommés `check`, `chm`, finissant en `.Rcheck`, `old` ou `old`, et les fichiers `GNUmakefile`, `Read-and-delete-me`, ou dont le nom commence par `.#`, ou finit par `~`, `.bak` ou `.swp`.

● Concept 8 — Le format Rd

DÉFINITION (§2.1).

« R objects are documented in files written in "R documentation" (Rd) format, a simple markup language much of which closely resembles (La)TeX, which can be processed into a variety of formats, including LaTeX, HTML and plain text. The translation is carried out by functions in the `tools` package called by the script `Rdconv`. »

« The R distribution contains more than 1400 such files which can be found in the `src/Library/pkg/man` directories. »

L'exemple du manuel — une version simplifiée de `load.Rd` :

```

\name{load}
\alias{load}
\title{Reload Saved Datasets}
\description{
  Reload datasets written with the function save.
}
\usage{
load(file, envir = parent.frame(), verbose = FALSE)
}
\arguments{
  \item{file}{a (readable binary-mode) connection
    or a character string giving the name of the file to load
    (when tilde expansion is done).}
  \item{envir}{the environment where the data should be loaded.}
  \item{verbose}{should item names be printed during loading?}
}
\value{
  A character vector of the names of objects created, invisibly.
}
\seealso{
  \link{save}.
}
\examples{
## save all data
save(list = ls(all.names = TRUE), file = "all.RData")
## restore the saved values to the current environment
load("all.RData")
}
\keyword{file}

```

La structure (§2.1). « *An Rd file consists of three parts. The header gives basic information about the name of the file, the topics documented, a title, a short textual description and R usage information for the objects documented. The body gives further information (for example, on the function's arguments and return value). Finally, there is an optional footer with keyword information. The header is mandatory.* »

« *Information is given within a series of sections with standard names (and user-defined sections are also allowed). Unless otherwise specified these should occur only once in an Rd file.* »

Les quatre champs obligatoires, tels que `R CMD check` les vérifie (point 11 du §1.3.1) : `\name` , `\alias` , `\title` , `\description` .

Comment reconnaître le type d'exercice

Le signal dans l'énoncé	Ce qu'il faut mobiliser
« créer la structure d'un paquet »	<code>package.skeleton()</code>
« quels fichiers sont obligatoires ? »	<code>DESCRIPTION</code> et <code>NAMESPACE</code>
« un sous-répertoire vide »	interdit — absent, oui ; vide, non
« ma version 0.9 est vue comme antérieure à 0.75 »	ce n'est pas un décimal
« quel champ pour cette dépendance ? »	<code>Imports</code> dans le cas normal
« l'utilisateur doit voir les fonctions de l'autre paquet »	<code>Depends</code>
« utilisé seulement dans les exemples »	<code>Suggests</code>
« j'ajoute des méthodes pour ses classes »	<code>Enhances</code>
« puis-je le mettre dans deux champs ? »	non — « <i>only one of these fields</i> »
« exporter une fonction »	<code>export(f)</code>
« exporter un nom non standard »	des guillemets sont obligatoires
« importer seulement deux fonctions »	<code>importFrom</code> , recommandé sur <code>import</code>
« ma méthode S3 n'est pas trouvée »	<code>S3method(generic, class)</code>
« faut-il exporter <code>print.foo</code> ? »	non , si <code>S3method</code> est présent
« faire quelque chose au chargement »	<code>.onLoad</code> , pas <code>.onAttach</code>
« afficher un message de démarrage »	<code>.onAttach</code> + <code>packageStartupMessage</code>
« charger du code compilé »	<code>useDynLib</code> ou <code>.onLoad</code>
« ça marche chez moi, pas sous <code>check</code> »	<code>--vanilla</code> → <code>~/R/check.Renviron</code>
« exclure un fichier du tarball »	<code>.Rbuildignore</code>
« documenter une fonction »	un fichier <code>.Rd</code> dans <code>man</code>
« des objets internes non documentés »	un fichier <code>pkg-internal.Rd</code> , ou ne pas les exporter

Comment résoudre ce type d'exercice

Protocole « créer un paquet » — 6 étapes.

1. `package.skeleton()` pour la structure, ou créer à la main : `DESCRIPTION` , `NAMESPACE` , `R/` , `man/` .
2. Remplir **les sept champs obligatoires** du `DESCRIPTION` , en **ASCII**, en respectant la forme de `Version` .
3. Mettre le code dans `R/` — **des assignations seulement**, aucun effet de bord, **pas de** `require` **ni d' options** .
4. Déclarer les dépendances **dans un seul champ**, en privilégiant `Imports` .
5. Écrire le `NAMESPACE` : `export` explicite, `importFrom` sélectif, `S3method` pour chaque méthode.
6. Documenter **chaque objet de niveau utilisateur** dans `man/` .

Protocole « livrer » — 4 étapes.

1. **Installer d'abord** : « *please check that your package can be installed ... you may get **more detailed error messages** doing the install directly* ».
2. `R CMD check` sur le répertoire, puis corriger.
3. `R CMD build` pour produire le tarball.
4. **Relancer** `check` **sur le tarball** — « *it is strongly recommended that the final checks are run on a tar archive prepared by* `R CMD build` ».

Protocole « déboguer un `check` qui échoue alors que tout marche » — 3 étapes.

1. Se rappeler que `check` **tourne en** `--vanilla` : ni `.Rprofile` , ni `.Renv` , ni `.RData` (fiche 300).
2. Si des paquets sont dans une bibliothèque non standard, **poser** `R_LIBS` **dans l'environnement** ou utiliser `~/R/check.Renv` .
3. Vérifier que **tout paquet utilisé** — `library` , `require` , `::` , `:::` , `NAMESPACE` — **est déclaré** dans `Depends` , `Imports` ou `Suggests` : c'est la vérification n° 5.

● Common mistakes

Erreur	Correction
Appeler un paquet une « bibliothèque »	« <i>A package is not a library</i> »
Dire qu'on « compile » un paquet	« <i>Compilation is not a correct term for a package</i> »
Laisser un sous-répertoire vide	absent, oui ; vide, non
Créer <code>foo</code> et <code>Foo</code>	fortement déconseillé — systèmes insensibles à la casse
Traiter <code>Version</code> comme un nombre	<code>0.9 < 0.75</code>
Terminer le <code>Title</code> par un point	« <i>not end in a period</i> »
Répéter le nom du paquet dans le <code>Title</code>	« <i>Do not repeat the package name</i> »
Commencer la <code>Description</code> par « This package »	« <i>good practice not to</i> »
Mettre plusieurs paragraphes dans <code>Description</code>	« <i>only one paragraph</i> »
Mettre une liste de diffusion en <code>Maintainer</code>	« <i>should be a person</i> »
Déclarer un paquet dans deux champs	« <i>only one of these fields</i> »
Mettre en <code>Depends</code> ce qui suffit en <code>Imports</code>	règle 2 du §1.1.3
Oublier de déclarer un paquet appelé par <code>::</code>	<code>check</code> le signale (point 5)
Déclarer <code>Depends: R</code> sans version	« <i>makes no sense</i> »
Dépendre d'un niveau de correctif non nul	« <i>inadvisable</i> »
Appeler <code>require</code> ou <code>options</code> dans <code>R/</code>	« <i>definitely should not</i> »
Croire le nom du fichier lié aux objets créés	« <i>there need be no connection</i> »
Utiliser des caractères non ASCII dans le code	« <i>will normally fail when the package is installed</i> »
Exporter une méthode S3 au lieu de l'enregistrer	<code>S3method</code> suffit et vaut mieux

Erreur	Correction
Utiliser <code>exportPattern</code> largement	« <i>not recommended for production code</i> »
Préférer <code>import</code> à <code>importFrom</code>	« <i>using importFrom selectively ... is good practice</i> »
Oublier d'exporter <code>.Last.lib</code>	le manuel le cite comme erreur fréquente
Charger du code compilé dans <code>.onAttach</code>	ce doit être <code>.onLoad</code> ou <code>useDynLib</code>
Afficher un message avec <code>cat</code> au chargement	<code>packageStartupMessage</code>
S'attendre à ce que <code>check</code> lise votre <code>.Rprofile</code>	<code>--vanilla</code>
Ne vérifier que le répertoire source	relancer <code>check</code> sur le tarball
Laisser des objets utilisateur non documentés	<code>check</code> le signale (point 12)
Nommer autrement que <code>value</code> le dernier argument d'une fonction de remplacement	<code>check</code> le signale (point 10)

Ultimate Review

Vocabulaire. Paquet source (les fichiers maîtres) → **tarball** → **paquet installé** (« *the result of running `R CMD INSTALL`* ») ; plus les **paquets binaires**, « *unpacked rather than installing from sources* ». « **A package is not a library** » — le mot désigne soit **le répertoire d'installation**, soit **la bibliothèque partagée du système**. (*macOS distingue `.so` et `.dyLib`.*)

Quatre opérations. **install** · **build** (« *creating a tarball ready for distribution, including cleaning it up and creating PDF/HTML documentation from any vignettes* ») · **check** (« *a test installation is done and tested ... consistency and portability* ») · « **compilation is not a correct term for a package** » — c'est le code C/C++/Fortran qui se compile, ou **le byte-compilage du code R**, « *enabled by default for all packages* ».

Charger contre attacher. « *Library performs both steps, but a package's namespace can be loaded without the package being attached* ». **Chargement paresseux** : « *the R objects ... are created at installation time and stored in a database ... loaded into the session at first use ... makes the R session start up faster and use less memory* ».

Structure. `DESCRIPTION` et `NAMESPACE`, plus `R data demo exec inst man po src tests tools vignettes` — « **some of which can be missing, but which should not be empty** » · fichiers optionnels `INDEX configure cleanup LICENSE NEWS` · **pas de fichiers cachés** · `package.skeleton` aide à démarrer.

Noms. Le répertoire porte **le nom du paquet** · **ne pas distinguer deux paquets par la casse** · caractères interdits " * : / < > ? \ | et les noms réservés Windows (con , prn , aux , nul , com1 - com9 , lpt1 - lpt9) · noms de .Rd **ASCII sans %** · **100 octets** de chemin, pour les tarballs · **pas de binaires exécutables** — « *CRAN will not accept submissions containing binary files even if they are listed* ».

DESCRIPTION . Format **Debian Control File** ; **noms de champs sensibles à la casse**, tous **capitalisés** ; continuation par **espace ou tabulation** ; **tout en ASCII**, sinon champ **Encoding** . **Sept champs obligatoires** : **Package** (≥ 2 caractères, commence par une lettre, ne finit pas par un point) · **Version** (au moins deux entiers, « *it is not a decimal number* » — 0.9 < 0.75) · **License** · **Title** (title case, 65 caractères, sans balisage, sans point final, ne répète pas le nom) · **Description** (un seul paragraphe, URL entre chevrons) · **Author** (« *all significant contributors must be included* ») · **Maintainer** (une personne, adresse entre chevrons). **Author** et **Maintainer** se déduisent d' **Authors@R** .

Dépendances. **Depends** — « *attached before the current package* » · **Imports** — « *namespaces are imported from ... but which do not need to be attached* », obligatoire pour tout **::** ou **:::** · **Suggests** — « *examples, demos, tests or vignettes* » et « *packages loaded in the body of functions* » · **Enhances** . **Trois règles** : (1) **un seul champ** ; (2) espace de noms seul → **Imports** ; (3) attachement nécessaire → **Depends** . « *It makes no sense to declare a dependence on R without a version specification, nor on the package base* ».

Sous-répertoire R . « *R code files, only* », extensions **.R** recommandée · « *must be possible to read in the files using source() , so R objects must be created by assignments* » · « *there need be no connection between the name of the file and the R objects created by it* » · « *definitely should not call functions with side effects such as require and options* » · **ASCII seulement** ; \uxxxx dans les chaînes · exceptions : **sysdata.rda** et les fichiers **.in** .

Sous-répertoire man . « *(only) documentation files ... in Rd format* » · « *all user-level objects in a package should be documented* » ; les objets internes → **pkg-internal.Rd** , ou **ne pas les exporter** · « *Having a man directory containing no documentation files may give an installation error* ».

NAMESPACE . « *although the file looks like R code ... it is not processed as R code* » · « *Namespaces are sealed once they are loaded ... imports and exports cannot be changed* » · recherche : **le paquet** → **les imports** → **base** → **le chemin normal** (« *so the base namespace precedes the normal search* ») · **export** (guillemets pour les noms non standard) · **exportPattern** (« *not recommended for production code* ») · **import** et **importFrom** (« *using importFrom selectively ... is good practice* ») · **le namespace de base** est importé implicitement.

Méthodes S3. « *The standard method for S3-style UseMethod dispatching might fail to locate methods defined in a package that is imported but not attached* » → **S3method(print, foo)** , qui « *ensures that the method is registered ... and the function print.foo does not need to be exported* » · troisième argument possible · **enregistrement différé** depuis R 3.6.0.

Points d'entrée. `.onLoad` (chargement) et `.onAttach` (attachement) — « *defined in the namespace but not exported* » · `.onDetach` / `.Last.lib` (celle-ci doit être exportée) · `.onUnload` · « *code to set options and load shared objects should be placed in a* `.onLoad` » · « *It is good practice for these functions to be quiet ...* `packageStartupMessage` » · `useDynLib` pour les objets partagés.

Vérifier et construire. « `R CMD check` and `R CMD build` run R processes with `--vanilla` » → `R_LIBS` dans l'environnement, ou `~/R/check.Renviron` · `TMPDIR` sans espace, slashes avant, système respectant la casse · « *the final checks are run on a tar archive prepared by* `R CMD build` » · `check` contrôle notamment la complétude du `DESCRIPTION`, les paquets non déclarés, la syntaxe R, le chargement avec `base` seul, les arguments des méthodes S3, le `value` des fonctions de remplacement, le `PACKAGE` des appels étrangers, les champs Rd obligatoires, les objets non documentés, les fins de ligne LF · `.Rbuildignore` pour exclure.

Format Rd. « *a simple markup language much of which closely resembles (La)TeX* », converti par `Rdconv` ; plus de 1400 fichiers dans la distribution · trois parties : en-tête (obligatoire), corps, pied de mots-clés (optionnel) · champs obligatoires `\name`, `\alias`, `\title`, `\description` · « *sections with standard names ... user-defined sections are also allowed ... should occur only once* ».

Active Recall

« *A package is a directory of files which extend R, a source package (the master files), or a tarball containing the files of a source package, or an installed package, the result of running `R CMD INSTALL`. On some platforms there are also binary packages ... which can be unpacked rather than installing from sources.* » (§1)

*« **A package is not a library.** The latter is used in two senses : a directory into which packages are installed ... [et] that used by the operating system, as a shared, dynamic or static library or ... a DLL, where the second L stands for "library". »*

« *although this is a persistent mis-usage. It seems to stem from S, whose analogues of R's packages were officially known as library sections ... but almost always referred to as libraries.* »

D'où l'ironie : la commande de chargement s'appelle `library()` .

« **Compilation is not a correct term for a package. Installing a source package which contains C, C++ or Fortran code will involve compiling that code. There is also the possibility of "byte" compiling the R code in a package (using the facilities of package compiler) : nowadays this is enabled by default for all packages. So compiling a package may come to mean byte-compiling its R code.** » (§1)

Deux choses différentes portent le même mot : la compilation du code **C/Fortran** du répertoire `src` , et le **byte-compilage** du code **R** — ce dernier étant désormais **automatique**.

Les opérations correctement nommées sont : **install, build, check**.

« **The sources of an R package consist of a subdirectory containing the files DESCRIPTION and NAMESPACE , and the subdirectories R , data , demo , exec , inst , man , po , src , tests , tools and vignettes (some of which can be missing, but which should not be empty).** » (§1.1)

Fichiers optionnels : `INDEX` , `configure` , `cleanup` , `LICENSE` , `LICENCE` , `NEWS` . Ignorés par R mais utiles : `INSTALL` , `README` , `ChangeLog` .

« **packages should not contain Unix-style "hidden" files/directories** » — sauf `.Rbuildignore` , `.Rinstignore` et `vignettes/.install_extras` .

Et `package.skeleton` « *can help to create the structure for a new package* ».

« **The Package , Version , License , Description , Title , Author , and Maintainer fields are mandatory, all other fields are optional. Fields Author and Maintainer can be auto-generated from Authors@R .** » (§1.1.1)

Le piège : « **The mandatory Version field ... is a sequence of at least two (and usually three) non-negative integers ... It is not a decimal number, so for example 0.9 < 0.75 since 9 < 75 .** »

Chaque composante est comparée comme un entier séparé. La version 0.9 précède la 0.75.

Et : « *a version such as 0.01 or 0.01.0 will be handled as if it were 0.1-0* ».

Les trois règles générales du §1.1.3 :

1. « *A package should be listed in only one of these fields.* »
2. « *Packages whose namespace only is needed to load the package using `library(pkgname)` should be listed in the Imports field and not in the Depends field.* »
3. « *Packages that need to be attached to successfully load the package ... must be listed in the Depends field.* »

La question de décision : l'utilisateur doit-il voir les fonctions de l'autre paquet ? Non → Imports (le cas normal). Oui → Depends .

Suggests couvre « *packages used only in examples, demos, tests or vignettes* » et « *packages loaded in the body of functions* ». Enhances : « *packages "enhanced" by the package at hand* ».

« *Namespaces accessed by the :: and ::: operators must be listed* » — et R CMD check le vérifie.

« *The R subdirectory contains R code files, only ... It should be possible to read in the files using `source()`, so R objects must be created by assignments.* » (§1.1.5)

« *Note that there need be no connection between the name of the file and the R objects created by it.* »

« Ideally, the R code files should only directly assign R objects and definitely should not call functions with side effects such as `require` and `options` . » Et : « Extreme care is needed if top-level computations are made to depend on availability or not of other packages ... Nor should they depend on the availability of external resources such as downloads. »

L'encodage : *« Only ASCII characters ... should be used ... Non-ASCII characters in object names will normally fail when the package is installed ... `\uxxxx` escapes should be used »* dans les chaînes.

Deux exceptions : `sysdata.rda` et les fichiers `.in` .

« Namespaces are sealed once they are loaded. Sealing means that imports and exports cannot be changed and that internal variable bindings cannot be changed. Sealing allows a simpler implementation strategy ... and allows code analysis and compilation tools to accurately identify the definition corresponding to a global variable reference in a function body. » (§1.5)

C'est la contrepartie technique de la garantie décrite en fiche 315 : un paquet ne peut pas être perturbé de l'extérieur **parce qu'il est verrouillé**.

La recherche : *« If not found locally, R searches the package namespace first, then the imports, then the base namespace and then the normal search path (so the base namespace precedes the normal search rather than being at the end of it). »*

Et : *« although the file looks like R code ... it is not processed as R code. Only very simple conditional processing of `if` statements is implemented. »*

« The standard method for S3-style `UseMethod` dispatching might fail to locate methods defined in a package that is imported but not attached to the search path. To ensure that these methods are available the packages defining the methods should ... register the methods using `S3method` directives. » (§1.5.2)

Le dispatch de la fiche 311 cherche `generic.class` par le nom. Si le paquet est **chargé sans être attaché** — le cas de tout `Imports` —, le nom n'est visible nulle part.

```
S3method(print, foo)
```

« *ensures that the method is registered and available for `UseMethod` dispatch, and the function `print.foo` does not need to be exported.* »

Cela dispense d'exporter la méthode — ce qui rejoint la recommandation du §1.1.5 : les objets internes qu'on n'exporte pas n'ont pas à être documentés.

Dans `bar` : `c <- function(...) sum(...)` et `g <- function(y) f(c(y, 7))` . Dans `foo` : `x <- 1` et `f <- function(y) c(x, y)` .

1. `g(6)` évalue `c(6, 7)` . **On est dans l'espace de noms de `bar`** , où `c` vaut `sum` → **13**.
2. `f(13)` : `f` vient de `foo` . **On est maintenant dans l'espace de noms de `foo`** , qui ne définit pas `c` — la recherche continue vers `base` , où `c` concatène.
3. `c(x, 13)` avec `x <- 1` de `foo` → `c(1, 13)` .

« *This is consistent with the definitions of `c` in the two settings : in `bar` the function `c` is defined to be equivalent to `sum` , but in `foo` the variable `c` refers to the standard function `c` in `base` .* » (§1.5.5)

Chaque fonction cherche ses noms depuis son propre paquet — la garantie vaut **entre paquets**, pas seulement contre l'utilisateur.

Et : « *Calling `library(bar)` loads `bar` and attaches its exports ... Package `foo` is also loaded but not attached* ».

« *Since loading and attaching are distinct operations, separate hooks are provided for each. These hook functions are called `.onLoad` and `.onAttach` ... they should be defined in the namespace but not exported.* » (§1.5.3)

Plus `.onDetach` ou `.Last.lib` au détachement, et `.onUnload` au déchargement.

Les erreurs énumérées : « *These hooks are often used incorrectly. People forget to export `.Last.Lib`. Compiled code should be loaded in `.onLoad` (or via a `useDynLib` directive) and unloaded in `.onUnLoad`. Do remember that a package's namespace can be loaded without the namespace being attached ... and that a package can be detached and re-attached whilst its namespace remains loaded.* »

« *Packages are not likely to need `.onAttach` (except perhaps for a start-up banner)* », et
« *It is good practice for these functions to be quiet. Any messages should use `packageStartupMessage`.* »

« *R CMD check and R CMD build run R processes with `--vanilla` in which none of the user's startup files are read.* » (§1.3)

C'est l'explication de la mise en garde de la fiche 300 : ni `.Rprofile`, ni `.Renviron`, ni `.RData`. Tout ce qui dépendait d'un `library()` posé dans un profil, d'un `R_LIBS` personnalisé ou d'un objet venant de `.RData` **n'existe pas**.

La solution donnée par le manuel : « *If you need `R_LIBS` set ... you can set it in the environment : also you can use the check and build environment files ...*

~/.R/check.Renviron and ~/.R/build.Renviron. »

Et une seconde cause fréquente : la vérification n° 5 exige que **tout paquet appelé** — par `library`, `require`, `::`, `:::` ou le `NAMESPACE` — **soit déclaré**.

Extraites de la liste du §1.3.1 :

- **Point 5** — « *all packages mentioned in `library` or `requires` or from which the `NAMESPACE` file imports or are called via `::` or `:::` are listed* » (fiche 315).
- **Point 10** — « *whether S3 methods have all the arguments of the corresponding generic* » (fiche 311) et « *whether the final argument of replacement functions is called `value`* » (fiche 302, §3.4.4).

- **Point 10** aussi — « *all foreign function calls (`.C` , `.Fortran` , `.Call` and `.External`) are tested to see if they have a `PACKAGE` argument* » (fiche 320).
- **Point 11** — « *the presence of the mandatory fields (`\name` , `\alias` , `\title` and `\description`)* ».
- **Point 12** — « *missing documentation entries, such as undocumented user-level objects* » (§1.1.5).

`check` est donc la mise en application automatique de règles disséminées dans tous les manuels.

1. « *Before using these tools, please check that your package can be installed. `R CMD check` will inter alia do this, but you may get more detailed error messages doing the install directly.* » (§1.3)
2. `R CMD check` sur le répertoire source.
3. `R CMD build` — « *It is recommended that packages are built for release by the current release version of R or "r-patched", to avoid inadvertently picking up new features of a development version.* »
4. `R CMD check` sur le tarball — « *It is strongly recommended that the final checks are run on a tar archive prepared by `R CMD build`.* »

Et : « *Run-time checks whether the package works correctly should be performed using `R CMD check` prior to invoking the final build procedure.* »

Pour exclure des fichiers : `.Rbuildignore` , « *Perl-like regular expressions ... one per line, to be matched case-insensitively* ».

« *An Rd file consists of three parts. The header gives basic information about the name of the file, the topics documented, a title, a short textual description and R usage information. The body gives further information (for example, on the function's arguments and return value). Finally, there is an optional footer with keyword information. The header is mandatory.* » (§2.1)

Sur l'exemple `load.Rd` : l'en-tête est `\name` , `\alias` , `\title` , `\description` , `\usage` ; le corps `\arguments` , `\value` , `\seealso` , `\examples` ; le pied `\keyword` .

« *Information is given within a series of sections with standard names (and user-defined sections are also allowed) ... should occur only once in an Rd file.* »

Le format est « *a simple markup language much of which closely resembles (La)TeX* », converti par **Rdconv** ; la distribution en contient **plus de 1400** exemples.

Flashcards

Question	Réponse
Combien de paquets dans la distribution R ?	Environ 30
Les quatre formes d'un paquet ?	Source, tarball, installé, binaire
Un paquet est-il une bibliothèque ?	Non
Les deux sens de « library » ?	Le répertoire d'installation · la bibliothèque partagée
D'où vient la confusion ?	De S — « <i>library sections</i> »
Que distingue macOS ?	<code>.so</code> et <code>.dylib</code>
Les trois opérations bien nommées ?	install, build, check
Pourquoi pas « compiler » ?	Ce terme n'est pas correct pour un paquet
Qu'est-ce qui est byte-compilé ?	Le code R , par défaut
<code>library()</code> fait combien d'étapes ?	Deux — charger et attacher
Peut-on charger sans attacher ?	Oui — par <code>pkg::fun</code>
Qu'est-ce que le chargement paresseux ?	Objets créés à l'installation , chargés au premier usage
Son bénéfice ?	Démarrage plus rapide, moins de mémoire
Les deux fichiers obligatoires ?	<code>DESCRIPTION</code> et <code>NAMESPACE</code>
Un sous-répertoire peut-il être absent ?	Oui — mais pas vide
Les fichiers cachés sont-ils permis ?	Non , sauf exceptions listées
Quelle fonction aide à démarrer ?	<code>package.skeleton</code>
Peut-on avoir <code>foo</code> et <code>Foo</code> ?	Fortement déconseillé
Longueur de chemin recommandée ?	100 octets
Les binaires exécutables ?	Interdits — et CRAN les refuse
Le format du <code>DESCRIPTION</code> ?	Un Debian Control File

Question	Réponse
Les noms de champs sont-ils sensibles à la casse ?	Oui — tous capitalisés
Comment continue-t-on une ligne ?	Par une espace ou une tabulation
Combien de champs obligatoires ?	Sept
Lesquels ?	<code>Package</code> <code>Version</code> <code>License</code> <code>Description</code> <code>Title</code> <code>Author</code> <code>Maintainer</code>
Lesquels peuvent être déduits ?	<code>Author</code> et <code>Maintainer</code> , d' <code>Authors@R</code>
Contraintes sur <code>Package</code> ?	Au moins 2 caractères , commence par une lettre , ne finit pas par un point
<code>Version</code> est-elle un décimal ?	Non — <code>0.9 < 0.75</code>
Longueur d'affichage du <code>Title</code> ?	Tronqué à 65 caractères
Le <code>Title</code> finit-il par un point ?	Non
Doit-il répéter le nom du paquet ?	Non
Combien de paragraphes pour <code>Description</code> ?	Un seul
Comment y écrire une URL ?	Entre chevrons
Le <code>Maintainer</code> peut-il être une liste de diffusion ?	Non — une personne
Que fait <code>Depends</code> ?	Les paquets sont attachés avant
Que fait <code>Imports</code> ?	L'espace de noms est importé, non attaché
Où déclarer un paquet appelé par <code>::</code> ?	<code>Imports</code> , <code>Suggests</code> ou <code>Enhances</code>
Que couvre <code>Suggests</code> ?	Exemples, démos, tests, vignettes
Et <code>Enhances</code> ?	Les paquets enrichis par le vôtre
Règle 1 des dépendances ?	Un seul champ par paquet
Règle 2 ?	Espace de noms seul → <code>Imports</code>

Question	Réponse
Règle 3 ?	Attachement nécessaire → <code>Depends</code>
Déclarer <code>Depends: R</code> sans version ?	N'a aucun sens
Que contient le répertoire <code>R</code> ?	Du code R, uniquement
Quelle extension recommandée ?	<code>.R</code>
Que doit permettre le fichier ?	D'être lu par <code>source()</code>
Le nom du fichier et les objets créés ?	Aucun lien nécessaire
Que ne doit-on pas y appeler ?	<code>require</code> et <code>options</code>
Quel encodage ?	ASCII
Deux exceptions dans <code>R/</code> ?	<code>sysdata.rda</code> et les fichiers <code>.in</code>
Que contient <code>man</code> ?	Les fichiers Rd , uniquement
Quels objets doivent être documentés ?	Tous ceux de niveau utilisateur
Où documenter les objets internes ?	<code>pkg-internal.Rd</code> — ou ne pas les exporter
Un <code>man</code> vide ?	Peut donner une erreur d'installation
Le <code>NAMESPACE</code> est-il du code R ?	Non
Que signifie « scellé » ?	Imports, exports et liaisons inchangeables
L'ordre de recherche d'un paquet ?	Paquet → imports → <code>base</code> → chemin normal
Que fait <code>export(f, g)</code> ?	Il exporte <code>f</code> et <code>g</code>
Quand faut-il des guillemets ?	Pour les noms non standard
Que fait <code>exportPattern("^[^.]")</code> ?	Il exporte tout ne commençant pas par un point
Est-ce recommandé ?	Pas en production
<code>import</code> ou <code>importFrom</code> ?	<code>importFrom</code> , sélectivement
Quel espace de noms est importé implicitement ?	<code>base</code>

Question	Réponse
Pourquoi enregistrer une méthode S3 ?	Le dispatch peut ne pas la trouver
Quelle directive ?	<code>S3method(generic, class)</code>
Faut-il exporter la méthode ?	Non
Le point d'entrée au chargement ?	<code>.onLoad</code>
À l'attachement ?	<code>.onAttach</code>
Au déchargement ?	<code>.onUnload</code>
Laquelle doit être exportée ?	<code>.Last.lib</code>
Où charger le code compilé ?	Dans <code>.onLoad</code> ou par <code>useDynLib</code>
Comment afficher un message ?	<code>packageStartupMessage</code>
Sous quelle option tournent <code>check</code> et <code>build</code> ?	<code>--vanilla</code>
Comment leur passer <code>R_LIBS</code> ?	Par l'environnement ou <code>~/R/check.Renviron</code>
Sur quoi lancer les vérifications finales ?	Le tarball produit par <code>build</code>
Comment exclure un fichier du tarball ?	<code>.Rbuildignore</code>
Qu'est-ce que le format Rd ?	Un langage de balisage proche de (La)TeX
Quel script le convertit ?	<code>Rdconv</code>
Combien de fichiers Rd dans la distribution ?	Plus de 1400
Les trois parties d'un fichier Rd ?	En-tête, corps, pied de mots-clés
Laquelle est obligatoire ?	L' en-tête
Les quatre champs vérifiés par <code>check</code> ?	<code>\name</code> · <code>\alias</code> · <code>\title</code> · <code>\description</code>